



Representação de Conhecimento

Programação em Lógica Estendida

Universidade do Minho

Inteligência Artificial em Engenharia Biomédica

1º Semestre – 22/23

26 de novembro de 2022

Mariana Ribeiro, A9496;

Mariana Almeida, A97380;

Raquel Lima, A97590;

Marta Ferreira, A95978.

Resumo

Este trabalho teve como objetivo a implementação de Programação em Lógica Estendida e, consequentemente, a implementação de Conhecimento Perfeito e Imperfeito a um sistema de representação de conhecimento e raciocínio capaz de caracterizar um universo de análises clínicas.

Para isso, foi utilizada a linguagem PROLOG, que apesar de apresentar limitações no que toca à representação de conhecimento negativo [Analide, Neves], através da abordagem da Programação em Lógica Estendida foi possível a representação de conhecimento não só perfeito negativo, mas também de conhecimento imperfeito.

Para obter uma caracterização do conhecimento imperfeito dos três tipos, foi necessária a realização de exceções e valores nulos. Por outro lado, para caracterizar o conhecimento negativo foi necessário definir o Pressuposto do Mundo Fechado [Analide, Neves]. Ainda foram desenvolvidos predicados que permitissem a evolução e involução da base de conhecimento, bem como a evolução do conhecimento.

Com o objetivo de tornar o sistema de representação mais realista foi necessário desenvolver sistemas de inferência que fossem capazes de responder com valores como falso, verdadeiro e desconhecido.

Tabela de Conteúdos

Resumo.....	2
Lista de Figuras	4
1. Introdução	7
2. Preliminares.....	7
2.1. Linguagem Prolog	7
2.2. Factos	8
2.3. Regras.....	8
2.4. Questões	8
2.5. Pressuposto de Mundo Fechado	8
2.6 Extensão à Programação em Lógica	8
2.7 Invariantes	9
2.8 Conhecimento Imperfeito: Valores Nulos.....	9
3. Descrição do trabalho e Análise de Resultados.....	9
3.1. Declarações iniciais.....	9
3.2. Base de conhecimento	10
4. Evolução da Base de Conhecimento	10
4.1. Invariantes de Inserção.....	11
5. Involução da Base de Conhecimento	15
5.1. Invariantes de Remoção	16
6. Sistemas de inferência.....	18
6.1 Sistema de Inferência Conjunção	20
6.2 Sistema de Inferência Disjunção	21
6.3 Sistema de Inferência Lista	22
7. Predicados auxiliares.....	23
7.1 Meta-predicado <i>não</i>	23
7.2 Predicado <i>comprimento</i>	23
7.3 Predicado <i>listinaoutente</i>	23
8. Conhecimento positivo.....	24
9. Conhecimento negativo.....	25
10. Conhecimento Imperfeito (Valores Nulos)	26
10.1 Conhecimento tipo I – Incerto.....	26
10.2 Conhecimento tipo II – Impreciso.....	27
10.3 Conhecimento tipo III – Interdito.....	29
11. Evolução do conhecimento	30

12. Relacionar análises e atos com o utente	34
12.1 Predicado relatar_atos_usuario	34
12.2 Predicados listagemcolesterol, listagempulsação e listagempressão.....	35
13. Marcadores	35
15. Conclusões	36
16. Referências Bibliográficas	37

Lista de Figuras

Figura 1 – Definições iniciais para a base de conhecimento primária.	9
Figura 2 – Extensão dos predicados: utente, ato e marcador.	10
Figura 3 – Extensão do predicado evolução.....	11
Figura 4 – Evolução de um novo utente.....	11
Figura 5 – Invariante inserir utente.	11
Figura 6 – Evolução de um utente já existente.....	12
Figura 7 – Invariante de inserção de um utente com id já existente.....	12
Figura 8 – Inserção de um utente num id já existente.	12
Figura 9 – Invariante que impede a inserção de utentes com menos de 18 anos.	13
Figura 10 – Exemplo da tentativa de evolução de um utente com menos de 18 anos.	13
Figura 11 – Exemplo de tentativa de evolução de utente com mais de 45 anos.	13
Figura 12 – Exemplo de tentativa de evolução de um utente com mais de 45 anos.	13
Figura 13 – Invariante para a inserção de um ato repetido.	13
Figura 14 – Tentativa de evolução de um ato já existente.	14
Figura 15 – Invariante que impede a inserção de um ato cujo utente não existe.	14
Figura 16 – Tentativa de evolução de um ato relacionado com um utente que não existe.	14
Figura 17 – Evolução de um ato associado a um utente já existente.	14
Figura 18 – Invariante que impede a inserção de pulsações menores ou iguais a zero.....	15
Figura 19 – Tentativa de evolução de um ato com pulsação nula.....	15
Figura 20 – Invariante que impede o registo de atos com valores de colesterol superiores a 600.	15
Figura 21 – Tentativa de evolução de um ato com colesterol a 660.	15
Figura 22 – Extensão do predicado involução.	16
Figura 23 – Remoção de um ato médico.....	16
Figura 24 – Invariante de remoção de um utente se ato associado.	16
Figura 25 – Involução de um utente com ato já existente.	17
Figura 26 – Involução de um utente com ato não existente.	17
Figura 27 – Invariante de remoção de um marcador se ato com idade dentro do marcador.	18

Figura 28 – Involução de um marcador com atos com idade dentro do marcador.....	18
Figura 29 – Sistema de Inferência.....	18
Figura 30 – Sistema de Inferência Conjunção.....	19
Figura 31 – Sistema de Inferência Disjunção.....	19
Figura 32 – Resposta do siC duas questões verdadeiras.	20
Figura 33 – Resposta do siC a uma questão verdadeira e outra falsa.....	20
Figura 34 – Resposta do siC a uma questão desconhecida e outra falsa.....	20
Figura 35 – Resposta do siD a uma questão verdadeira e outra verdadeira. Erro! Marcador não definido.	
Figura 36 – Resposta do siD a uma questão verdadeira e outra verdadeira.....	20
Figura 37 – Resposta do siD a uma questão verdadeira e outra falsa.....	21
Figura 38 – Resposta do siD a uma questão falsa e outra verdadeira.....	21
Figura 39 – Resposta do siD a uma questão verdadeiro e outra desconhecida.....	22
Figura 40 – Sistema de Inferência Lista.....	22
Figura 41 – Resposta do sistema de Inferência Lista a uma lista de duas questões verdadeiras. 22	
Figura 42 – Resposta do sistema de Inferência Lista a uma lista de duas questões desconhecida e verdadeira.....	23
Figura 43 – Negação por falha.....	23
Figura 44 – Predicado comprimento.....	23
Figura 45 – Extensão do predicado listingnaoutente.....	23
Figura 46 – Conhecimento perfeito positivo da base de conhecimento.....	24
Figura 47 – Conhecimento perfeito positivo.....	24
Figura 48 – Conhecimento perfeito negativo dos utentes.....	25
Figura 49 – Conhecimento perfeito negativo do predicado utente.....	25
Figura 50 – Conhecimento perfeito negativo relativo ao predicado marcador.....	25
Figura 51 – Conhecimento perfeito negativo para o predicado marcador.....	26
Figura 52 – Valor nulo incerto do predicado utente.....	26
Figura 53 – Reposta do sistema de inferência do predicado utente a um valor nulo incerto.....	26
Figura 54 – Valor nulo incerto do predicadoato.....	26
Figura 55 – Reposta do sistema de inferência do predicado ato a um valor nuloincerto.....	27
Figura 56 – Valor nulo impreciso no predicadoutente.....	27
Figura 57 – Resposta do sistema de inferência no predicado utente a um valor nulo impreciso... ..	27
Figura 58 – Valor nulo impreciso no predicado utente.....	28
Figura 59 – Resposta do sistema de inferência no predicado utente a um valor nulo impreciso.....	28
Figura 60 – Valor nulo impreciso no predicado ato.....	28

Figura 61 – Resposta do sistema de inferência no predicado ato a um valor nulo impreciso.....	29
Figura 62 – Valor nulo interdito no predicado utente.....	29
Figura 63 – Resposta do sistema de inferência no predicado utente a um valor nulo interdito..	30
Figura 64 - Extensão do predicado alterar_sexo.....	30
Figura 65 – Evolução do utente com id “456123”.....	31
Figura 66 – Alteração do sexo do utente com id “456123”.....	31
Figura 67 – Extensão do predicado alterar_sexo.....	31
Figura 68 – Evolução do utente com id "1245".....	32
Figura 69 – Alteração do nome do utente com id “1245”.....	32
Figura 70 – Extensão do predicado alterar_data_ato.....	32
Figura 71 – Alteração da data de um ato com id “gmr2183”.....	33
Figura 72 – Predicado alterarnaoutente.....	33
Figura 73 – Teste do predicado alterarnaoutente.....	33
Figura 74 – Predicado alterarutente.....	34
Figura 75 – Teste do predicado alterarutente.....	34
Figura 76 – Predicado relatar_atos_utente.....	34
Figura 77 – Teste do predicado relatar_atos_utente.....	35
Figura 78 – Predicado listagemcolesterol.....	35
Figura 79 – Predicadolistagempulsacao.....	35
Figura 80 – Predicadolistagempressao.....	35
Figura 81 – Testes dos predicados listagemcolesterol, listagempulsacao elistagempressao.....	35
Figura 82 – Predicado dentropulsacao.....	36
Figura 83 – Predicado forapulsacao.....	36
Figura 84 – Teste dos predicados dentropulsacao e forapulsacao.....	36

1. Introdução

Como já foi referido anteriormente, o objetivo do trabalho foi a implementação de Conhecimento Perfeito e Imperfeito a um sistema de representação de conhecimento e raciocínio capaz de caracterizar um universo de análises clínicas.

De forma geral, era pedido a criação de um sistema que fosse capaz de:

- Representar conhecimento positivo e negativo
- Representar casos de conhecimento imperfeito, pela utilização de valores nulos de todos os tipos estudados;
- Representar invariantes que designem restrições à inserção e à remoção de conhecimento;
- Lidar com a problemática da evolução do conhecimento, criando os procedimentos adequados;
- Desenvolver um sistema de inferência capaz de implementar os mecanismos de raciocínio inerentes a estes sistemas;
- Relatar análises e exames por utente/ato.

Assim, neste relatório serão apresentados os predicados desenvolvidos, bem como os invariantes e exceções para a representação do conhecimento.

No geral, o relatório apresenta o raciocínio levado em conta no desenvolvimento dos predicados e cláusulas necessários ao funcionamento do sistema de representação de conhecimento.

O relatório explicita uma parte com os conhecimentos prévios necessários à realização do trabalho, na secção Preliminares, posteriormente, apresenta uma secção com Descrição do Trabalho e Análise dos Resultados, seguida a explicitação de todo os predicados desenvolvidos.

2. Preliminares

As linguagens tradicionais de Programação em Lógica proporcionam uma ferramenta poderosa para a representação de conhecimento. Neste trabalho, irá utilizar-se a linguagem em Prolog, uma das mais usadas.

2.1. Linguagem Prolog

O PROLOG é uma linguagem declarativa em que existem apenas respostas de verdadeiro ou falso, uma vez que implementa os fundamentos da lógica matemática clássica [Analide, Neves]. O programa consiste em dados baseados num conjunto de cláusulas – factos, regras e questões.

2.2. Factos

Os factos expressam algo que é sempre verdadeiro. Por exemplo, a extensão do predicado utente (“O utente António é do sexo masculino”) irá ser representada da seguinte forma:

utente (antonio, masculino).

2.3. Regras

As regras expressam algo que é verdadeiro, dependente da veracidade das condições. Por exemplo, “O Antonio é utente se o seu id for “123456780”

utente(antonio) :- id(123456780).

2.4. Questões

O utilizador pode averiguar a veracidade das situações, fazendo questões à base de conhecimento. Deste modo, a questão “O utente António é de sexo masculino?” será representada da seguinte forma:

? - utente(antonio,masculino).

Na qual o programa irá responder *true* ou *false*, dependendo da base de conhecimento. No entanto, se se quiser conhecer o sexo do utente “António”, a questão iria ser alterada para:

?- utente(antonio,X)

Neste caso, a resposta obtida seria o valor de X:

X=masculino?

true

2.5. Pressuposto de Mundo Fechado

As generalidades dos programas escritos em Lógica representam implicitamente a informação negativa, assumindo a aplicação do raciocínio segundo o Pressuposto de Mundo Fechado.

O Pressuposto de Mundo Fechado declara que toda a informação que não existe na Base de Conhecimento é considerada falsa [Analide, Neves]. Assim, conhecimento pode ser equacionado apenas em termos de verdadeiro ou falso, não permitindo abordar a representação de informação incompleta.

2.6 Extensão à Programação em Lógica

A extensão de um programa em Lógica passa a contar com dois tipos de negação: a negação por falha (representativa de sistema tradicionais e retratada pelo termo *nao*), e a negação forte, como forma de identificar informação falsa (negativa), simbolizada por $\neg$ [Analide, Neves].

Em sistemas construídos segundo esta forma de extensão de programação em lógica, passa-se a ter uma terceira hipótese de resposta, a *desconhecida*, se nenhuma das conclusões anteriores for possível.

Deste forma, formaliza-se o PMF de modo a contemplar estas situações, da seguinte forma genérica:

$$\neg p(X): -nao(p(X)), nao(excecao(p(X))).$$

2.7 Invariantes

Para que haja coerência na base de conhecimento, é necessário assegurar um conjunto de condições ou restrições, denominadas de invariantes. Estes podem ser do tipo estrutural ou referencial.

Os invariantes estruturais impedem, por exemplo, a inserção de conhecimento repetido. Já os invariantes referenciais lidam com informação completa, mas não com informação inconsistente.

2.8 Conhecimento Imperfeito: Valores Nulos

Existem três tipos de valores nulos que podem surgir numa situação de informação incompleta.

Valor Nulo Incerto: trata-se de um valor desconhecido, genérico onde não existe necessário um conjunto de hipóteses.

Valor Nulo Impreciso: trata-se de um valor desconhecido. No entanto, neste caso, existe um conjunto de hipóteses dentro do qual se encontra a informação precisa.

Valor Nulo Interdito: trata-se de um valor desconhecido e não permitido de conhecer.

3. Descrição do trabalho e Análise de Resultados

Nesta secção do relatório será apresentado uma introdução ao trabalho desenvolvido.

3.1. Declarações iniciais

Inicialmente, é necessário definir para cada predicado a declaração *dynamic* como se pode observar pela figura 1. Esta declaração serve para poder alterar variáveis estáticas na base de conhecimento primária tendo em conta o número de argumentos de cada predicado.

```
:-dynamic (utente/4).
:-dynamic (ato/8).
:-dynamic (marcador/6).
:- op(900, xfy, '::~').
:- dynamic ('-/1).
:- op(900, xfy, 'e').
:- op(900, xfy, 'ou').
```

Figura 1 – Definições iniciais para a base de conhecimento primária.

Assim, através destas declarações, as variáveis inicialmente estáticas passam a ser dinâmicas e podem ser alteradas por outros predicados. Então, a base de conhecimento pode ser alterada, sendo possível introduzir-se ou remover-se conhecimento representado pelos predicados indicados.

Também se declaram operadores como ':' para poder introduzir invariantes à inserção ou remoção de conhecimento. Outros exemplos, são os operadores 'e' e 'ou' utilizados nos sistemas de inferência Conjunção e Disjunção explicados mais a frente neste trabalho.

3.2. Base de conhecimento

A base de conhecimento foi definida com conhecimento positivo e negativo através da inserção de factos dos predicados *utente*, *ato* e *marcador*.

```
% Extensao do predicado utente: IdUt, Nome, (Dia, Mes, Ano), Sexo -> { V,F,D }  
% Extensao do predicado ato: IdAto, (Dia, Mes, Ano), IdUt, Idade, Colesterol, Pulsacao, PInf, PSup -> { V,F,D }  
% Extensao do predicado marcador: IdMarcador, Analise, Idade, Sexo, VMinimo, VMaximo -> { V,F,D }
```

Figura 2 – Extensão dos predicados: *utente*, *ato* e *marcador*.

O predicado *utente*, tem como argumentos o *Id*, *Nome do utente*, *Data* na forma de tuplo (*Dia*, *Mês*, *Ano*) e *Sexo* do utente.

Por sua vez, o predicado *ato*, tem como argumentos *Id* do ato, *Data* na forma (*Dia*, *Mês*, *Ano*), *Id* do utente, *Idade*, *Colesterol*, *Pulsação*, *Pressão Arterial Inferior* e *Pressão Arterial Superior*.

Por último, o predicado *marcador* com o *Id* do marcador, o tipo de *Análise* efetuada, a *Idade*, *Sexo*, e o *Valor* mínimo e máximo aceitáveis para essa análise.

Todos estes predicados têm como contradomínio o conjunto { V,F,D } quando utilizado o sistema de inferência desenvolvido, que irá ser apresentado mais a frente no relatório.

A declaração dos factos está também apresentada mais a frente no relatório na secção correspondente à declaração do conhecimento positivo e também negativo.

4. Evolução da Base de Conhecimento

O predicado *evolução* é permite a inserção dos registos de utentes, atos e marcadores na base de conhecimento. Com o intuito de se inserir um termo verifica-se, com o auxílio do predicado *findall*, se existem invariantes relacionados com o termo em questão, criando-se uma lista com esses mesmo, em caso afirmativo. Posteriormente, a lista é testada, e na ocorrência de uma falha, retira-se o termo que foi adicionado pelo predicado inserção através da função *retract*.

```
% Extensao do predicado que permite a evolucao do conhecimento

evolucao(Termo) :-
    findall(Invariante, +Termo::Invariante, Lista),
    insercao(Termo),
    teste(Lista).

insercao(Termo) :-assertz(Termo).
insercao(Termo) :-retract(Termo),!, fail.

teste([]).
teste([R|LR]):- R, teste(LR).
```

Figura 3 – Extensão do predicado evolução.

Por exemplo, adicionando a utente “Gabriela” com id “1345”, nascida a 14-04-2000, do sexo *feminino*, irá ser observado o seguinte no interpretador:

```
?- evolucao(utente(1345,gabriela,(14,4,2000),feminino)).
true
```

Figura 4 – Evolução de um novo utente.

De modo a garantir uma maior fiabilidade dos dados inseridos na base de conhecimento, recorre-se à utilização de invariantes.

Os invariantes são regras de teste à consistência da Base de Conhecimento e podem ser, como referido na secção Preliminares, de dois tipos: estruturais, ou seja, impedem a inserção de conhecimento repetido, e referenciais que lidam com informação completa, mas não com informação inconsistente.

4.1. Invariantes de Inserção

4.1.1. Invariantes para o predicado utente

Através do invariante da figura 5, impede-se o **registo de um utente já existente** na base de conhecimento.

```
+utente(IdUt, Nome, (Dia, Mes, Ano), Sexo) :: (findall((IdUt, Nome), (utente( IdUt, Nome, (Dia, Mes, Ano), Sexo)), S),
comprimento( S,N ),
N == 1 ).
```

Figura 5 – Invariante inserir utente.

Por exemplo, se o utilizador desejar inserir novamente o utente “António” de id “123456780”, nascido a 30-06-1990, do sexo *masculino*, que já existe na Base de Conhecimento, não irá ser possível adicionar, tal como demonstrado na figura 6.

```
?- listing(utente).
:- dynamic utente/4.

utente(123456780, antonio, (30, 6, 1990), masculino).
utente(987654321, beatriz, (30, 7, 1985), feminino).
utente(135246978, carlos, (30, 8, 1987), masculino).
utente(111133333, raquel, (14, 10, 2002), feminino).
utente(222223333, sofia, (3, 7, 1989), feminino).
utente(333334444, helena, (4, 2, 1990), feminino).
utente(444445555, marta, (15, 3, 1995), feminino).
utente(555556666, diana, (13, 4, 2000), feminino).
utente(666667777, joana, (21, 12, 1988), feminino).
utente(111222333, silvio, xpto1, masculino).
utente(555666777, gabriel, (17, 2, 2000), xpto3).

true.

?- evolucao(utente(123456780, antonio, (30, 6, 1990), masculino)).
false.
```

Figura 6 – Evolução de um utente já existente.

Para **não permitir o registo de um utente com um IdUtente existente** na base de conhecimento, gerou-se o seguinte invariante (Figura 7).

+utente(IdUt, _ , _ , _) :: (findall((IdUt),(utente(IdUt, _ , _ , _)), S),			
			comprimento(S,N),
			N == 1).

Figura 7 – Invariante de inserção de um utente com id já existente.

A título de exemplo, caso o utente deseje registar o utente “Hélio”, nascido a 20-06-1980, do sexo masculino e com o id “123456780”, já existente na base de conhecimento, verifica-se que não é possível a inserção do mesmo (Figura 8).

```
?- listing(utente).
:- dynamic utente/4.

utente(123456780, antonio, (30, 6, 1990), masculino).
utente(987654321, beatriz, (30, 7, 1985), feminino).
utente(135246978, carlos, (30, 8, 1987), masculino).
utente(111133333, raquel, (14, 10, 2002), feminino).
utente(222223333, sofia, (3, 7, 1989), feminino).
utente(333334444, helena, (4, 2, 1990), feminino).
utente(444445555, marta, (15, 3, 1995), feminino).
utente(555556666, diana, (13, 4, 2000), feminino).
utente(666667777, joana, (21, 12, 1988), feminino).
utente(111222333, silvio, xpto1, masculino).
utente(555666777, gabriel, (17, 2, 2000), xpto3).

true.

?- evolucao(utente(123456780, helio, (20, 6, 1980), masculino)).
false.
```

Figura 8 – Inserção de um utente num id já existente.

Admitiu-se que a clínica apenas pode registar pacientes entre os 18 e 45 anos.

Desta forma, procedeu-se à criação de um invariante que **não permite a inserção de utentes nascidos após 2004**.

```
+utente(_, _, (_, _, A), _) :: (findall((A), (utente(_, _, (_, _, A), _), A > 2004), L),
                                comprimento(L, N),
                                N == 0).
```

Figura 9 – Invariante que impede a inserção de utentes com menos de 18 anos.

Com efeito, ao tentar adicionar a utente “Mariana” com *id* “2222222”, de sexo *feminino* e nascida a 21-05-2005, esta não será inserida (Figura 10).

```
?- evolucao(utente(2222222, mariana, (21,05,2005), feminino)).
false.
```

Figura 10 – Exemplo da tentativa de evolução de um utente com menos de 18 anos.

De uma forma análoga, construiu-se a invariante que **impede a inserção de utentes nascidos antes de 1977** (Figura 11).

```
+utente(_, _, (_, _, A), _) :: (findall((A),
                                         (utente(_, _, (_, _, A), _), A < 1977),
                                         L),
                                comprimento(L, N),
                                N == 0).
```

Figura 11 – Exemplo de tentativa de evolução de utente com mais de 45 anos.

Exemplificando, ao tentar adicionar a utente “Mariana”, com *id* “12838832”, de sexo *feminino* e nascida a 30-06-1954, esta não será inserida (Figura 12).

```
?- evolucao(utente(12838832, mariana, (30,06,1954), feminino)).
false.
```

Figura 12 – Exemplo de tentativa de evolução de um utente com mais de 45 anos.

4.1.2. Invariantes para o predicado *ato*

Com vista a inserir um novo ato na base de conhecimento, elaborou-se um invariante que **impede a adição de um ato repetido** (Figura 13). Isto implica a impossibilidade de haver dois atos com *id* igual.

```
+ato(IdAto, _, _, _, _, _) :: (findall((IdAto), (ato(IdAto, _, _, _, _, _), S),
                                         comprimento(S, N),
                                         N == 1).
```

Figura 13 – Invariante para a inserção de um ato repetido.

Admitindo que se tenta inserir novamente o ato “gmr02067”, ao utente com *id* “123456780”, a resposta ao processo será a representada na Figura 14.

```
ato(gmr02067, (30, 6, 2020), 123456780, 30, 140, 70, 60, 123).
ato(gmr2183, (30, 7, 2021), 987654321, 36, 190, 60, 70, 142).
ato(gmr2297, (30, 6, 2022), 123456780, 32, 230, 90, 65, 151).
ato(gmr1111, (12, 9, 2020), 111133333, 18, 260, 90, 50, 80).
ato(gmr22223, (1, 1, 2019), 222223333, 30, 160, 170, 200, 80).
ato(gmr22223, (2, 3, 2018), 333334444, 28, 140, 120, 80, 310).
ato(gmr02000, (18, 11, 2022), 4545456666, 30, xpto2, 70, 60, 123).

true.

?- evolucao(ato(gmr02067, (30, 6, 2020), 123456780, 30, 140, 70, 60, 123)).
false.
```

Figura 14 – Tentativa de evolução de um ato já existente.

De forma a **não permitir a inserção de um ato relacionado a um utente não existente**, criou-se o seguinte invariante (Figura 15).

```
+ato(_, _, IdUt, _, _, _, _) :: (findall((IdUt), (ato(_, _, IdUt, _, _, _, _)), (utente(IdUt, _, _))), S),
comprimento(S,N),
N>=1).
```

Figura 15 – Invariante que impede a inserção de um ato cujo utente não existe.

Analisando a Figura 15, verifica-se que não é possível inserir o ato “gmr6666”, uma vez que não existe o utente com *id* “1111111”.

```
?- evolucao(ato(gmr6666, (10, 05, 2019), 1111111, 40, 130, 60, 70, 130)).
false.
```

Figura 16 – Tentativa de evolução de um ato relacionado com um utente que não existe.

Pelo contrário, a funcionalidade de registar um ato com um utente existente na base de conhecimento funciona corretamente (Figura 17), já que não viola nenhum dos invariantes desenvolvidos.

```
utente(123456780, antonio, (30, 6, 1990), masculino).
utente(987654321, beatriz, (30, 7, 1985), feminino).
utente(135246978, carlos, (30, 8, 1987), masculino).
utente(111133333, raquel, (14, 10, 2002), feminino).
utente(222223333, sofia, (3, 7, 1989), feminino).
utente(333334444, helena, (4, 2, 1990), feminino).
utente(444445555, marta, (15, 3, 1995), feminino).
utente(555556666, diana, (13, 4, 2000), feminino).
utente(666667777, joana, (21, 12, 1988), feminino).
utente(111222333, silvio, xpto1, masculino).
utente(555666777, gabriel, (17, 2, 2000), xpto3).

true.

?- evolucao(ato(gmr7, (30, 8, 2020), 123456780, 30, 140, 70, 60, 123)).
true ■
```

Figura 17 – Evolução de um ato associado a um utente já existente.

Existem valores críticos que se devem ter em conta aquando do resultado das análises feitas a um utente. Para tal, foram criados alguns invariantes que tiveram em atenção certos valores, nomeadamente a pulsação e o colesterol. É evidente, por exemplo, que a pulsação de um utente deve ser maior que zero, caso contrário, a pessoa estaria morta.

Desta forma, gerou-se um invariante que **impede a inserção de pulsações com valores inferiores ou iguais a 0**. (Figura 18).

```
+ato( _, _, _, _, Pul, _, _ ) :: (findall((Pul),(ato( _, _, _, _, Pul, _, _ ), Pul=<0), L),
                                comprimento(L,N),
                                N==0).
```

Figura 18 – Invariante que impede a inserção de pulsações menores ou iguais a zero.

Através da observação da figura 19, verifica-se que não é possível inserir o ato “gmr1111”, cuja pulsação é 0.

```
?- evolucao(ato(gmr1111, (12,03,2019), 987654321, 34, 180,0, 70, 142)).
false.
```

Figura 19 – Tentativa de evolução de um ato com pulsação nula.

O desenvolvimento do invariante representado na figura 20, **não permitirá que o colesterol esteja a mais ou igual a 600**, fora dos valores considerados possíveis.

```
+ato( _, _, _, _, Col, _, _ ) :: (findall((Col),
                                (ato( _, _, _, _, Col, _, _ ), Col>600),
                                L),
                                comprimento(L,N),
                                N==0).
```

Figura 20 – Invariante que impede o registo de atos com valores de colesterol superiores a 600.

A título de exemplo, é possível observar que não é possível a evolução do ato “gmr3” cujo colesterol está a 660 (Figura 21).

```
?- evolucao(ato(gmr3, (30, 7, 2021), 987654321, 36, 660, 60, 70, 142)).
false.
```

Figura 21 – Tentativa de evolução de um ato com colesterol a 660.

5. Involução da Base de Conhecimento

A *involução* consiste num predicado que tem o objetivo contrário do predicado *evolução*, ou seja, este permite a remoção de registos de utentes, atos e marcadores da base de conhecimento.

A fim de remover um termo verifica-se, com o auxílio do predicado *findall*, se existem invariantes relacionados com o termo em questão, criando-se uma lista com esses. Posteriormente, a lista é testada, e na ocorrência de uma falha o processo de remoção não é concluído, caso contrário, o termo é removido da Base de Conhecimento.

```

involucao( Termo ) :-
    findall(Invariante, -Termo::Invariante, Lista),
    remocao(Termo),
    teste(Lista).

remocao(Termo):-retract(Termo).
remocao(Termo):-assertz(Termo), !, fail.

teste([]).
teste([R|LR]):-R, teste(LR).
  
```

Figura 22 – Extensão do predicado involução.

Por exemplo, removendo um ato com *id* “gmr02067”, realizado a 30-06-2020, associado ao utente de *id* “123456780” com 30 anos de idade e cujos valores de colesterol é 140, pulsação 70 e a pressão encontra-se entre 60 e 123, irá ser observado o seguinte no interpretador.:

```

?- involucao(ato(gmr02067, (30,06,2020), 123456780, 30, 140, 70, 60, 123)).
true
  
```

Figura 23 – Remoção de um ato médico.

Novamente, de forma a garantir uma maior consistência dos dados na base de conhecimento, recorre-se à utilização de invariantes.

5.1. Invariantes de Remoção

5.1.1. Invariantes para o predicado utente

Através do invariante da figura 24, impede-se a remoção de um utente com um ato já associado.

```

-utente(IdUt,_,_,_)::(findall((IdUt),(ato(_,_,IdUt,_,_,_,_,_)), S),
    comprimento( S,N ),
    N == 0 ).
  
```

Figura 24 – Invariante de remoção de um utente se ato associado.

Por exemplo, pretende-se a remoção do utente *António* de *id* “123456780”, nascido a 30-06-1990, do sexo *masculino*, não irá ser possível eliminar da base de conhecimento, tal como demonstrado na figura 25 pois, já existem atos de *id* “gmr02067” e “gmr2297” associados a este utente.


```
?- listing(utente).
:- dynamic utente/4.

utente(123456780, antonio, (30, 6, 1990), masculino).
utente(987654321, beatriz, (30, 7, 1985), feminino).
utente(135246978, carlos, (30, 8, 1987), masculino).
utente(111133333, raquel, (14, 10, 2002), feminino).
utente(222233333, sofia, (3, 7, 1989), feminino).
utente(333334444, helena, (4, 2, 1990), feminino).
utente(444445555, marta, (15, 3, 1995), feminino).
utente(555556666, diana, (13, 4, 2000), feminino).
utente(666667777, joana, (21, 12, 1988), feminino).
utente(111222333, silvio, xpto1, masculino).
utente(555666777, gabriel, (17, 2, 2000), xpto3).

true.

?- listing(ato).
:- dynamic ato/8.

ato(gmr02067, (30, 6, 2020), 123456780, 30, 140, 70, 60, 123).
ato(gmr2183, (30, 7, 2021), 987654321, 36, 190, 60, 70, 142).
ato(gmr2297, (30, 6, 2022), 123456780, 32, 230, 90, 65, 151).
ato(gmr1111, (12, 9, 2020), 111133333, 18, 260, 90, 50, 80).
ato(gmr22223, (1, 1, 2019), 222233333, 30, 160, 170, 200, 80).
ato(gmr22223, (2, 3, 2018), 333334444, 28, 140, 120, 80, 310).
ato(gmr02000, (18, 11, 2022), 4545456666, 30, xpto2, 70, 60, 123).

true.

?- involucao(utente(123456780, antonio, (30, 6, 1990), masculino)).
false.
```

Figura 25 – Involução de um utente com ato já existente.

Pelo contrário, através do invariante da figura 24, permite-se a remoção de um utente que não tenha ato associado, por exemplo a *Marta* com *id* “444445555” nascida em 15-3-1995 como verificado na figura seguinte.

```
?- listing(utente).
:- dynamic utente/4.

utente(987654321, beatriz, (30, 7, 1985), feminino).
utente(135246978, carlos, (30, 8, 1987), masculino).
utente(111133333, raquel, (14, 10, 2002), feminino).
utente(222233333, sofia, (3, 7, 1989), feminino).
utente(333334444, helena, (4, 2, 1990), feminino).
utente(444445555, marta, (15, 3, 1995), feminino).
utente(555556666, diana, (13, 4, 2000), feminino).
utente(666667777, joana, (21, 12, 1988), feminino).
utente(111222333, silvio, xpto1, masculino).
utente(555666777, gabriel, (17, 2, 2000), xpto3).
utente(123456780, antonio, (30, 6, 1990), masculino).

true.

?- listing(ato).
:- dynamic ato/8.

ato(gmr02067, (30, 6, 2020), 123456780, 30, 140, 70, 60, 123).
ato(gmr2183, (30, 7, 2021), 987654321, 36, 190, 60, 70, 142).
ato(gmr2297, (30, 6, 2022), 123456780, 32, 230, 90, 65, 151).
ato(gmr1111, (12, 9, 2020), 111133333, 18, 260, 90, 50, 80).
ato(gmr22223, (1, 1, 2019), 222233333, 30, 160, 170, 200, 80).
ato(gmr22223, (2, 3, 2018), 333334444, 28, 140, 120, 80, 310).
ato(gmr02000, (18, 11, 2022), 4545456666, 30, xpto2, 70, 60, 123).

true.

?- involucao(utente(444445555, marta, (15, 3, 1995), feminino)).
true.
```

Figura 26 – Involução de um utente com ato não existente.

5.1.2. Invariantes para o predicado marcador

Através do invariante da figura 27, impede-se a remoção de um marcador se existirem atos com idade dentro do marcador.

```
-marcador(IdMarc, Nome, (IMin, IMax), Sexo, Vmin, Vmax)::(findall((Idade), (ato(IdAto, (DataN), IdUt, Idade, Col, Pul, PInf, PSup), Idade<IMax, Idade>IMin), L),
comprimento(L,N),
N==0).
```

Figura 27 – Invariante de remoção de um marcador se ato com idade dentro do marcador.

Por exemplo, pretende-se a remoção do marcador “ctf06” com idades entre 46 e 50 anos e sendo que não há nenhum ato com valores de idade entre esse intervalo, o mesmo poderá ser removido. No caso do marcador “ctm01” com um intervalo de idade entre 18 e 30 anos, este não irá ser possível eliminar da base de conhecimento, tal como demonstrado na figura 28 devido à existência de atos entre essas idades.

```
?- listing(marcador).
:- dynamic marcador/6.

marcador(ctm01, colesterol, (18, 30), masculino, 0, 170).
marcador(ctf02, colesterol, (18, 30), feminino, 0, 160).
marcador(ctm03, colesterol, (31, 45), masculino, 0, 190).
marcador(ctf04, colesterol, (31, 45), feminino, 0, 180).
marcador(psm05, pulsacao, (18, 25), masculino, 60, 80).
marcador(ctf06, colesterol, (46, 50), feminino, 0, 180).

true.

?- listing(ato).
:- dynamic ato/8.

ato(gmr02067, (30, 6, 2020), 123456780, 30, 140, 70, 60, 123).
ato(gmr2183, (30, 7, 2021), 987654321, 36, 190, 60, 70, 142).
ato(gmr2297, (30, 6, 2022), 123456780, 32, 230, 90, 65, 151).
ato(gmr1111, (12, 9, 2020), 111133333, 18, 260, 90, 50, 80).
ato(gmr22223, (1, 1, 2019), 222233333, 30, 160, 170, 200, 80).
ato(gmr22223, (2, 3, 2018), 333334444, 28, 140, 120, 80, 310).
ato(gmr02000, (18, 11, 2022), 4545456666, 30, xpto2, 70, 60, 123).

true.

?- involucao(marcador(ctf06, colesterol, (46,50), feminino, 0, 180)).
true.

?- involucao(marcador(ctm01, colesterol, (18, 30), masculino, 0, 170)).
false.
```

Figura 28 – Involução de um marcador com atos com idade dentro do marcador.

6. Sistemas de inferência

De forma, a lidar com situações de informação incompleta, é necessário ter em conta não só os valores *falso* e *verdadeiro* mas também o valor *desconhecido*. Inicialmente, foi desenvolvido um sistema de inferência que pudesse responder a questões colocadas com valores como *verdadeiro*, *falso* e *desconhecido* de acordo com o conhecimento existente na base de conhecimento, como podemos verificar na figura 29:

```
si(Questao, verdadeiro) :-
    Questao.
si(Questao, falso) :-
    ~Questao.
si(Questao, desconhecido) :-
    nao(Questao),
    nao(~Questao).
```

Figura 29 – Sistema de Inferência.

Pode perguntar-se ao *si* perguntas básicas, ou seja, colocar uma só questão como, podemos observar na figura 53.

No entanto, pode ser necessário fazer questões mais complexas, como por exemplo a conjunção ou disjunção de duas questões diferentes. E, ainda, se for pretendida uma única resposta correspondente à associação das respostas singulares do sistema de inferência “normal” às questões, é necessário um outro sistema de inferência que faça a conjunção ou disjunção dessas respostas/questões.

Assim, foram desenvolvidos o *siC* (sistema de inferência capaz de realizar a conjunção de questões) e o *siD* (sistema de inferência capaz de realizar a disjunção de questões), como se pode observar nas figuras 30 e 31.

```

siC(Q1 e Q2, verdadeiro) :-
    si(Q1, verdadeiro),
    si(Q2, verdadeiro).
siC(Q1 e Q2, falso) :-
    si(Q1, verdadeiro),
    si(Q2, falso).
siC(Q1 e Q2, desconhecido) :-
    si(Q1, verdadeiro),
    si(Q2, desconhecido).
siC(Q1 e Q2, falso) :-
    si(Q1, falso),
    si(Q2, verdadeiro).
siC(Q1 e Q2, falso) :-
    si(Q1, falso),
    si(Q2, falso).
siC(Q1 e Q2, falso) :-
    si(Q1, falso),
    si(Q2, desconhecido).
siC(Q1 e Q2, desconhecido) :-
    si(Q1, desconhecido),
    si(Q2, verdadeiro).
siC(Q1 e Q2, falso) :-
    si(Q1, desconhecido),
    si(Q2, falso).
siC(Q1 e Q2, desconhecido) :-
    si(Q1, desconhecido),
    si(Q2, desconhecido).
  
```

```

siD(Q1 ou Q2, verdadeiro) :-
    si(Q1, verdadeiro),
    si(Q2, verdadeiro).
siD(Q1 ou Q2, verdadeiro) :-
    si(Q1, verdadeiro),
    si(Q2, falso).
siD(Q1 ou Q2, verdadeiro) :-
    si(Q1, verdadeiro),
    si(Q2, desconhecido).
siD(Q1 ou Q2, verdadeiro) :-
    si(Q1, falso),
    si(Q2, verdadeiro).
siD(Q1 ou Q2, falso) :-
    si(Q1, falso),
    si(Q2, falso).
siD(Q1 ou Q2, desconhecido) :-
    si(Q1, falso),
    si(Q2, desconhecido).
siD(Q1 ou Q2, verdadeiro) :-
    si(Q1, desconhecido),
    si(Q2, verdadeiro).
siD(Q1 ou Q2, desconhecido) :-
    si(Q1, desconhecido),
    si(Q2, falso).
siD(Q1 ou Q2, desconhecido) :-
    si(Q1, desconhecido),
    si(Q2, desconhecido).
  
```

Figura 30 – Sistema de Inferência Conjunção.

Figura 31 – Sistema de Inferência Disjunção

Estes sistemas de inferência capazes de realizar a conjunção e disjunção dependem do sistema de inferência “normal”, já que a conjunção ou disjunção das questões depende, como já referido acima, das respostas do sistema de inferência às questões colocadas separadamente.

Assim, foi necessário criar tabelas de verdade que incorporasse os 3 valores possíveis. Contruiui-se as seguintes tabelas para a conjunção e disjunção de valores.

Disjunção ($\vee$)		Questão1		
		V	F	D
Questão 2	V	V	V	V
	F	V	F	D
	D	V	D	D

Tab. 1 – Tabela de verdade da disjunção de valores verdadeiro, falso e desconhecido.

Conjunção ($\wedge$)		Questão1		
		V	F	D
Questão 2	V	V	F	D
	F	F	F	F
	D	D	F	D

Tab. 1 – Tabela de verdade da conjunção de valores verdadeiro, falso e desconhecido.

6.1 Sistema de Inferência Conjunção

Desta forma, para verificar se, por exemplo, existem os dados: *utente* (123456780, antonio, (30,06,1990), masculino) e *ato*(gmr02067, (30,06,2020), 123456780, 30, 140, 70, 60, 123), pode fazer-se a seguinte pergunta ao siC:

```
?- siC(utente(123456780, antonio, (30, 6, 1990), masculino) e ato(gmr02067,
(30, 6, 2020), 123456780, 30, 140, 70, 60, 123), R).
R = verdadeiro .
```

Figura 32 – Resposta do siC duas questões verdadeiras.

A resposta obtida é *verdadeiro*, porque, de facto, estes valores existem na base de conhecimento.

Desta forma, se for pretendido fazer a questão “Existe algum utente com *id* 123456780, chamado António que tenha um ato com colesterol 1000?” deve ser usado o sistema de inferência conjunção da seguinte forma:

```
?- siC(utente(123456780, antonio, DataN, Sexo)) e ato(Id, Dataato, 123456780,
I, 1000, Pul, Pinf, Psup), R).
DataN = (30, 6, 1990),
Sexo = masculino,
R = falso .
```

Figura 33 – Resposta do siC a uma questão verdadeira e outra falsa.

A resposta que se obtém é *falso*, pois apesar de existir um utente com este id e nome António e com os dados para *DataN* e *Sexo* obtidos, não existe nenhum ato com os valores introduzidos.

No exemplo a seguir, mostra-se como o siC responde *falso* à conjunção de uma questão *desconhecida* (já que o utente de *id* 111222333 tem um valor nulo do tipo I no parâmetro da data) e de uma questão *falsa* (já que não existe nenhum ato com os valores inseridos abaixo).

```
?- siC(utente(111222333, silvio, (12,12,2001), masculino)) e (ato(gmr2183,(30,
07,2021), 111222333, 36, 190,60,70,142)), R).
R = falso .
```

Figura 34 – Resposta do siC a uma questão desconhecida e outra falsa.

Finalmente, quando as duas questões perguntadas ao sistema *siC* são, respetivamente, *desconhecido* e *verdadeiro*, o sistema responde com *desconhecido*, de acordo com a tabela de verdade desenvolvida.

```
?- siC(utente(111222333, silvio, (12,12,2001), masculino)) e (ato(gmr2183, (30,07,2021), 987654321, 36,190,60,70,142)), R).
R = desconhecido
```

Figura 35 – Resposta do *siC* a uma questão desconhecida e outra verdadeira.

6.2 Sistema de Inferência Disjunção

De forma análoga, pode fazer-se a disjunção de questões colocadas ao sistema. Se por exemplo, o objetivo for colocar a seguinte questão “Existe o *ato(gmr02067, (30, 6, 2020), 123456780, 30, 140, 70, 60, 123)* ou o *ato(gmr22223, (1, 1, 2019), 222223333, 30, 160, 170, 200, 80)*?” deve utilizar-se o *siD* da seguinte forma:

```
?- siD(ato(gmr02067, (30, 6, 2020), 123456780, 30, 140, 70, 60, 123) ou ato(gmr22223, (1, 1, 2019), 222223333, 30, 160, 170, 200, 80), R).
R = verdadeiro
```

Figura 36 – Resposta do *siD* a uma questão verdadeira e outra verdadeira.

Como as duas questões colocadas são *verdadeiras*, o sistema responde com a conjunção destes dois valores, ou seja, *verdadeiro*, como se pode verificar na figura 36 acima.

Por outro lado, quando as questões colocadas são respetivamente *verdadeiro* e *falso*, o sistema responde *verdadeiro*, novamente. Neste caso, apesar das informações introduzidas relativas à data do ato serem falsas, a disjunção do valor falso com o valor verdadeiro da questão relativa ao utente, corresponde ao valor *verdadeiro*.

```
?- siD(utente(987654321, beatriz, (30, 7, 1985), feminino) ou ato(gmr22223, (01, 02, 1500), 222223333, 30, 160, 170, 200, 80), R).
R = verdadeiro .
```

Figura 37 – Resposta do *siD* a uma questão verdadeira e outra falsa.

No próximo exemplo, mostra-se a resposta do *siD* a uma questão falsa e desconhecida, a resposta esperada é *desconhecido*.

Como pode ver-se pela figura 38, o sistema responde como esperado.

```
?- siD(utente(555556666, cristiano, (13, 4, 2000), masculino) ou utente(111444, marcelo, (13,04,2000), masculino), R).
R = desconhecido.
```

Figura 38 – Resposta do *siD* a uma questão falsa e outra verdadeira.

Por fim, a disjunção de valores *verdadeiro* e *desconhecido*, corresponde ao valor *verdadeiro*. Para verificar que o sistema responde de acordo, pode fazer-se a seguinte pergunta: “Existe o *utente(333334444, helen, (4, 2, 1990), feminino)* ou o *utente(111222333, silvio, (21,12,2000), masculino)*?”

```
?- siD(utente(333334444, helen, (4, 2, 1990), feminino) ou utente(111222333,
silvio, (21,12,2000), masculino), R).
R = verdadeiro .
```

Figura 39 – Resposta do siD a uma questão verdadeiro e outra desconhecida.

Neste caso, é *desconhecido* se o *utente Sílvia* tem a data de nascimento correspondente a 21-12-2000, no entanto, é *verdadeiro* a existência da *utente helen* com os dados introduzidos. Assim, o sistema responde como esperado, com o valor *verdadeiro*.

6.3 Sistema de Inferência Lista

Por último também foi desenvolvido um sistema de inferência Lista, mostrado na figura 40, que recebendo uma lista de questões, devolve uma lista de respostas. Mais uma vez, este sistema de inferência Lista, utiliza o sistema de inferência base para responder a cada questão separadamente, e por fim, colocar as respostas numa lista que será devolvida.

```
siL([], []).
siL([Questao|L],[Resposta|S]):-
    si(L,S),
    si(Questao,Resposta).
```

Figura 40 – Sistema de Inferência Lista.

Na figura 41, pode observar-se o teste deste sistema de inferência quando introduzidas duas questões positivas.

```
?- siL([utente(123456780, antonio, (30, 6, 1990), masculino), utente(222223333,
sofia, (3, 7, 1989), feminino)],L).
L = [verdadeiro, verdadeiro]
```

Figura 41 – Resposta do sistema de Inferência Lista a uma lista de duas questões verdadeiras.

Por outro lado, verifica-se, na figura 42, que o sistema responde com uma lista com os valores *desconhecido* e *verdadeiro* quando são introduzidas questões, respetivamente, *desconhecida*, neste caso é desconhecida a data de nascimento do *utente Sílvia*, e *verdadeira*, estando de acordo com o sistema de inferência base.

```
?- siL([utente(111222333, silvio, (21,12,2000), masculino), utente(666667777, joana,
(21, 12, 1988), feminino)], L).
L = [desconhecido, verdadeiro] .
```

Figura 42 – Resposta do sistema de Inferência Lista a uma lista de duas questões desconhecida e verdadeira.

7. Predicados auxiliares

7.1 Meta-predicado *não*

O meta-predicado *não* permite a implementação da negação por falha. Assim, existe apenas um argumento (*Questao*), onde se pode obter duas respostas possíveis, verdadeiro ou falso.

Se o resultado dessa questão for verdadeiro, irá ocorrer falha na prova. Se for negativa a resposta, o meta-predicado retorna verdadeiro.

```
nao(Questao) :-
    Questao, !, fail.
nao(Questao).
```

Figura 43 – Negação por falha.

7.2 Predicado *comprimento*

O predicado *comprimento* foi desenvolvido como um auxílio à construção de invariantes.

Este predicado possui dois argumentos: uma lista (L) e a variável N, que irá ser o comprimento dessa lista. Através do uso da função *length* foi possível a sua extensão.

```
comprimento([],0).
comprimento( L,N ) :-
    length( L,N ).
```

Figura 44 – Predicado *comprimento*.

7.3 Predicado *listingnaoutente*

Devido à impossibilidade de listar conhecimento perfeito negativo dos utentes, foi desenvolvido o predicado *listingnaoutente*, que através de *findall* devolve numa lista toda a informação correspondente à parte negativo do predicado utente.

```
listingnaoutente(L):- findall((utente(IdUt, Nome , (Dia, Mes, Ano), Sexo)),
    (-utente(IdUt, Nome , (Dia, Mes, Ano), Sexo)),L).
```

Figura 45 – Extensão do predicado *listingnaoutente*.

8. Conhecimento positivo

Como é possível verificar na figura 46, foi definida a parte positiva da base de conhecimento através de factos com informação dada no enunciado.

```
% Extensao do predicado utente: IdUt, Nome, (Dia, Mes, Ano), Sexo -> { V,F,D }

utente(123456780, antonio, (30,06,1990), masculino).
utente(987654321, beatriz, (30,07,1985), feminino).
utente(135246978, carlos, (30,08,1987), masculino).

utente(111133333, raquel, (14,10,2002), feminino).
utente(222233333, sofia, (3,07,1989), feminino).
utente(333334444, helena, (4,02,1990), feminino).
utente(444445555, marta, (15,03, 1995), feminino).
utente(555556666, diana, (13,04,2000), feminino).
utente(666667777, joana, (21,12,1988), feminino).

% Extensao do predicado ato: IdAto, (Dia, Mes, Ano), IdUt, Idade, Colesterol, Pulsacao, PInf, PSup -> { V,F,D }

ato(gmr02067, (30,06,2020), 123456780, 30, 140, 70, 60, 123).
ato(gmr2183, (30,07,2021), 987654321, 36, 190, 60, 70, 142).
ato(gmr2297, (30,06,2022), 123456780, 32, 230, 90, 65, 151).

ato(gmr1111, (12,9,2020), 111133333, 18, 260, 90, 50, 80).
ato(gmr22223, (01,01,2019), 222233333, 30, 160, 170, 200, 80).
ato(gmr22223, (02,03,2018), 333334444, 28, 140, 120, 80, 310).

% Extensao do predicado marcador: IdMarcador, Analise, Idade, Sexo, VMinimo, VMaximo -> { V,F,D }

marcador(ctm01, colesterol, (18,30), masculino, 0, 170).
marcador(ctf02, colesterol, (18,30), feminino, 0, 160).
marcador(ctm03, colesterol, (31,45), masculino, 0, 190).
marcador(ctf04, colesterol, (31,45), feminino, 0, 180).
marcador(psm05, pulsacao, (18, 25), masculino, 60, 80).
```

Figura 46 – Conhecimento perfeito positivo da base de conhecimento.

Através do uso do predicado *listing* é possível listar todo o conhecimento perfeito positivo existente na base de conhecimento.

```
?- listing(utente).
:- dynamic utente/4.

utente(123456780, antonio, (30, 6, 1990), masculino).
utente(987654321, beatriz, (30, 7, 1985), feminino).
utente(135246978, carlos, (30, 8, 1987), masculino).
utente(111133333, raquel, (14, 10, 2002), feminino).
utente(222233333, sofia, (3, 7, 1989), feminino).
utente(333334444, helena, (4, 2, 1990), feminino).
utente(444445555, marta, (15, 3, 1995), feminino).
utente(555556666, diana, (13, 4, 2000), feminino).
utente(666667777, joana, (21, 12, 1988), feminino).
utente(111222333, silvio, xptol, masculino).
utente(555666777, gabriel, (17, 2, 2000), xpto3).

true.

?- listing(ato).
:- dynamic ato/8.

ato(gmr02067, (30, 6, 2020), 123456780, 30, 140, 70, 60, 123).
ato(gmr2183, (30, 7, 2021), 987654321, 36, 190, 60, 70, 142).
ato(gmr2297, (30, 6, 2022), 123456780, 32, 230, 90, 65, 151).
ato(gmr1111, (12, 9, 2020), 111133333, 18, 260, 90, 50, 80).
ato(gmr22223, (1, 1, 2019), 222233333, 30, 160, 170, 200, 80).
ato(gmr22223, (2, 3, 2018), 333334444, 28, 140, 120, 80, 310).
ato(gmr02000, (18, 11, 2022), 4545456666, 30, xpto2, 70, 60, 123).

true.

?- listing(marcador).
:- dynamic marcador/6.

marcador(ctm01, colesterol, (18, 30), masculino, 0, 170).
marcador(ctf02, colesterol, (18, 30), feminino, 0, 160).
marcador(ctm03, colesterol, (31, 45), masculino, 0, 190).
marcador(ctf04, colesterol, (31, 45), feminino, 0, 180).
marcador(psm05, pulsacao, (18, 25), masculino, 60, 80).

true.
```

Figura 47 – Conhecimento perfeito positivo.

9. Conhecimento negativo

Através do uso do predicado *listingnaoutente*, explicado mais detalhadamente nos predicados auxiliares, é possível listar o conhecimento perfeito negativo, em relação ao utente, existente na base de conhecimento.

```
?- listingnaoutente(L).
L = [utente(1928238723, julio, (22, 4, 1990), masculino), utente(9999888777, carolina, (12, 4, 2001), feminino)].
```

Figura 48 – Conhecimento perfeito negativo dos utentes.

Analogamente, foi também representado o conhecimento perfeito negativo através da criação de casos hipotéticos.

Caso I: O utente “Júlio” com id “1928238723”, nascido a 22-04-1990, de sexo *masculino* e o utente “Carolina”, nascida a 12-04-2002, do sexo *feminino* não estão inscritos na base de conhecimento (Figura 49).

```
-utente(1928238723, julio, (22,04,1990), masculino).
-utente(9999888777, carolina, (12,04,2001), feminino).
```

Figura 49 – Conhecimento perfeito negativo do predicado utente.

Caso II: Não existem marcadores para utentes que queiram fazer análises à *glicose* e à *hormonaX* (Figura 50).

```
-marcador(gl01, glicose, (18,45), feminino, 40, 100).
-marcador(hr02, hormonaX, (18,45), feminino, 30, 60).
```

Figura 50 – Conhecimento perfeito negativo relativo ao predicado marcador.

Caso III: A clínica não possui marcadores de análises clínicas para menores de 18 anos (Figura 51).

```
-marcador(ctm03, colesterol, (0,18), masculino, 0, 170).
-marcador(ctf03, colesterol, (0,18), feminino, 0, 170).
```

Figura 51 – Conhecimento perfeito negativo para o predicado marcador.

10. Conhecimento Imperfeito (Valores Nulos)

10.1 Conhecimento tipo I – Incerto

O Valor nulo tipo I trata-se de um conhecimento em que se representa o conhecimento desconhecido. Com a finalidade de testar este valor nulo, procedeu-se à elaboração de diferentes tipos de situações que eventualmente possam surgir.

Situação 1

O utente de *id* “111222333” e de nome *Sílvia* chega à clínica visivelmente perturbado e não sabe a sua data de nascimento.

Deste modo, a situação é representada da seguinte forma (Figura 52):

```
utente(111222333, silvio, xpto1, masculino).
execcao(utente(IdUt, Nome, (Dia, Mes, Ano), Sexo)):-utente(IdUt, Nome, xpto1, Sexo).
```

Figura 52 – Valor nulo incerto do predicado utente.

Com o auxílio do sistema de inferência, foi possível colocar a questão no interpretador, obtendo-se a seguinte resposta (Figura 53).

```
?- si(utente(111222333, silvio, xpto1, masculino), R).
R = verdadeiro .

?- si(utente(111222333, silvio, (10,10,2010), masculino), R).
R = desconhecido.
```

Figura 53 – Reposta do sistema de inferência do predicado utente a um valor nulo incerto.

Situação 2

Devido a um erro ocorrido nas análises clínicas do utente com *id* “4545456666”, a 18-11-2022, não foi possível determinar o valor do parâmetro colesterol no ato de *id* “gmr02000”.

Como tal, foi implementada a seguinte exceção para qualquer ato que apresente o colesterol desconhecido (xpto2), figura 54.

```
ato(gmr02000, (18,11,2022), 4545456666, 30, xpto2, 70, 60, 123).
execcao(ato( IdAto, Data, IdUt, Idade, Colesterol, Pulsacao, PInf, PSup)):-
ato(IdAto, Data, IdUt, Idade, xpto2, Pulsacao, PInf, PSup).|
```

Figura 54 – Valor nulo incerto do predicado *ato*.

Com o auxílio do sistema de inferência, foi possível colocar a questão no interpretador, obtendo-se a seguinte resposta (Figura 55).

```
?- si(ato(gmr02000,(18,11,2022),4545456666,30,xpto2,70,60,123),R).
R = verdadeiro .

?- si(ato(gmr02000,(18,11,2022),4545456666,30,100,70,60,123),R).
R = desconhecido.
```

Figura 55 – Resposta do sistema de inferência do predicado *ato* a um valor nulo incerto.

10.2 Conhecimento tipo II – Impreciso

De modo semelhante, foram criadas situações específicas numa tentativa de representar conhecimento imperfeito desconhecido impreciso em relação ao predicado *utente* e ao predicado *ato*.

Situação I

O utente de *id* “111222444”, do sexo *masculino* e nascido a 13-01-1998, chega à clínica apresentando alguns sintomas de demência, nomeadamente no que toca ao seu nome, não sabendo se se chama *Raul* ou *Renato*.

A representação deste conhecimento pode ser redigida através da figura 56.

```
excecao(utente(111222444,raul,(13,01,1988),masculino)).
excecao(utente(111222444,renato,(13,01,1988),masculino)).
```

Figura 56 – Valor nulo impreciso no predicado *utente*.

Com o auxílio do sistema de inferência, foi possível verificar a exceção no interpretador, obtendo-se as seguintes respostas (Figura 57).

```
?- si(utente(111222444,renato,(13,01,1988),masculino), R).
R = desconhecido.

?- si(utente(111222444,raul,(13,01,1988),masculino), R).
R = desconhecido.

?- si(utente(111222444,ruir,(13,01,1988),masculino), R).
R = falso ■
```

Figura 57 – Resposta do sistema de inferência no predicado *utente* a um valor nulo impreciso.

Situação II

A utente *Lúcia* de id “222333444”, do sexo *feminino*, sabe o mês e o ano em que nasceu, *12-1990*, mas não se recorda o dia ao certo, sabendo apenas que nasceu na semana do Natal, entre 23 a 27.

```
excecao(utente(222333444,lucia,(23,12,1990),feminino)).
excecao(utente(222333444,lucia,(24,12,1990),feminino)).
excecao(utente(222333444,lucia,(25,12,1990),feminino)).
excecao(utente(222333444,lucia,(26,12,1990),feminino)).
excecao(utente(222333444,lucia,(27,12,1990),feminino)).
```

Figura 58 – Valor nulo impreciso no predicado utente.

Com o auxílio do sistema de inferência, foi possível verificar a exceção no interpretador, obtendo-se as seguintes respostas (Figura 59).

```
?- si(utente(222333444,lucia,(22,12,1990),feminino),R).
R = falso .
?- si(utente(222333444,lucia,(23,12,1990),feminino),R).
R = desconhecido.
?- si(utente(222333444,lucia,(24,12,1990),feminino),R).
R = desconhecido.
?- si(utente(222333444,lucia,(25,12,1990),feminino),R).
R = desconhecido.
?- si(utente(222333444,lucia,(26,12,1990),feminino),R).
R = desconhecido.
?- si(utente(222333444,lucia,(27,12,1990),feminino),R).
R = desconhecido.
?- si(utente(222333444,lucia,(28,12,1990),feminino),R).
R = falso ■
```

Figura 59 – Resposta do sistema de inferência no predicado utente a um valor nulo impreciso.

Situação III

O utente de id “444555666” que deu entrada na clínica no dia 23-05-2022 apresenta todos os dados do ato fiáveis à exceção da pulsação, uma vez que ocorreu um erro na medição da mesma e não se sabe ao certo se o valor correto é 60 ou 62.

```
excecao(ato(gmr02001,(23,05,2022),444555666,35,160,60,61,122)).
excecao(ato(gmr02001,(23,05,2022),444555666,35,160,62,61,122)).
```

Figura 60 – Valor nulo impreciso no predicado ato.

Com o auxílio do sistema de inferência, foi possível verificar a exceção no interpretador, obtendo-se as seguintes respostas (Figura 61).

```
?- si(ato(gmr02001,(23,05,2022),444555666,35,160,63,61,122), R).
R = falso.

?- si(ato(gmr02001,(23,05,2022),444555666,35,160,62,61,122), R).
R = desconhecido.

?- si(ato(gmr02001,(23,05,2022),444555666,35,160,60,61,122), R).
R = desconhecido.
```

Figura 61 – Resposta do sistema de inferência no predicado ato a um valor nulo impreciso.

10.3 Conhecimento tipo III – Interdito

Com o pretexto de caracterizar conhecimento imperfeito sob a forma de conhecimento desconhecido cuja descoberta é sempre impossível, foi utilizado valores nulos do tipo III – interdito.

Analogamente aos restantes tipos de conhecimento, foram propostas situações com o fim de representar o conhecimento.

Situação I

O utente *Gabriel* de id “555666777”, nascido no dia 17-02-2000, recusa-se a revelar o seu sexo, pelo que este parâmetro se encontra bloqueado sendo impossível o seu conhecimento.

Foi também adicionado um invariante que assegura que nunca se introduzam valores para o parâmetro *Sexo*.

```
utente(555666777,gabriel,(17,02,2000), xpto3).
excecao(utente(IdUt, Nome , DataN, Sexo)):- utente(IdUt, Nome , DataN, xpto3).
interdito(xpto3).

+utente(IdUt, Nome , DataN, Sexo)::(findall((Sexo),
      (utente(555666777,gabriel,(17,02,2000),Sexo),nao(interdito(Sexo))),
      S),
      comprimento(S,N),
      N==0).
```

Figura 62 – Valor nulo interdito no predicado utente.

Com o auxílio do sistema de inferência, foi possível obter as seguintes respostas (Figura 63), verificando-se que através do invariante criado, é impossível introduzir informação no parâmetro *sexo*.

```

?- si(utente(555666777,gabriel,(17,02,2000), xpto3), R).
R = verdadeiro ,

?- si(utente(555666777,gabriel,(17,02,2000), feminino), R).
R = desconhecido.

?- si(utente(555666777,gabriel,(17,02,2000), masculino), R).
R = desconhecido.

?- evolucao(utente(555666777,gabriel,(17,02,2000), feminino)).
false.

```

Figura 63 – Resposta do sistema de inferência no predicado *utente* a um valor nulo interdito.

11. Evolução do conhecimento

A evolução do conhecimento é uma temática complexa, devendo ser lidada com cautela.

Por exemplo, existem inúmeras possibilidades de evolução de um tipo de conhecimento já existente para outro, nomeadamente de conhecimento negativo para positivo; impreciso para positivo; positivo para negativo, etc.

Deste modo, foi decidido que o conhecimento antigo seria apagado da base de conhecimento quando se inserisse o posterior através de predicados específicos.

A título de exemplo, foram implementados ao predicado *Utente*, a extensão do predicado *alterarutente* que permite evoluir conhecimento positivo para negativo, e a extensão do predicado *alterarnaoutente* que permitir passar de conhecimento negativo para positivo.

Foram também construídos predicados para alterar informações tais como o nome do utente, o sexo e a data de criação do ato, sendo este um exemplo de evolução de conhecimento positivo para positivo. Esta ferramenta é de extrema importância pois permite alterar informação que possa estar incorreta ou que já não seja verdadeira, garantindo assim uma melhor consistência da base de conhecimento. Com efeito, foram desenvolvidos o predicado *alterar_sexo*, *alterar_nome_utente* e *alterar_data_ato*.

11.1 Predicado *alterar_sexo* de um utente

Para o utilizador poder alterar o sexo do utente, foi necessária a extensão do predicado evidenciado na figura 64.

```

alterar_sexo(IdUt, Nome, Sexo):- involucao(utente(IdUt, Nome ,(Dia, Mes, Ano), _ )),
                                evolucao(utente(IdUt, Nome , (Dia, Mes, Ano), Sexo)).

```

Figura 64 - Extensão do predicado *alterar_sexo*.

A utilização deste predicado é útil, por exemplo, para a substituição do parâmetro sexo, caso tenha ocorrido um erro.

Tomando o exemplo da utente “Mariana” de *id* “456123”, nascida a 18-07-2002 que é inserida no sistema com sexo *masculino*:

```
?- evolucao(utente(456123,mariana,(18,07,2002),masculino)).
true.

?- listing(utente).
:- dynamic utente/4.

utente(123456780, antonio, (30, 6, 1990), masculino).
utente(987654321, beatriz, (30, 7, 1985), feminino).
utente(135246978, carlos, (30, 8, 1987), masculino).
utente(111133333, raquel, (14, 10, 2002), feminino).
utente(222223333, sofia, (3, 7, 1989), feminino).
utente(333334444, helena, (4, 2, 1990), feminino).
utente(444445555, marta, (15, 3, 1995), feminino).
utente(555556666, diana, (13, 4, 2000), feminino).
utente(666667777, joana, (21, 12, 1988), feminino).
utente(111222333, silvio, xpto1, masculino).
utente(555666777, gabriel, (17, 2, 2000), xpto3).
utente(456123, mariana, (18, 7, 2002), masculino).

true.
```

Figura 65 – Evolução do utente com *id* “456123”.

O sexo inserido não corresponde à realidade, visto que a utente é do sexo *feminino*. Deste modo, utiliza-se o *alterar_sexo*, inserindo o *id*, o nome do utente e o dado que se pretende alterar, verificando-se na figura 66, o sucesso na alteração da informação.

```
?- alterar_sexo(456123, mariana,feminino).
true.

?- listing(utente).
:- dynamic utente/4.

utente(123456780, antonio, (30, 6, 1990), masculino).
utente(987654321, beatriz, (30, 7, 1985), feminino).
utente(135246978, carlos, (30, 8, 1987), masculino).
utente(111133333, raquel, (14, 10, 2002), feminino).
utente(222223333, sofia, (3, 7, 1989), feminino).
utente(333334444, helena, (4, 2, 1990), feminino).
utente(444445555, marta, (15, 3, 1995), feminino).
utente(555556666, diana, (13, 4, 2000), feminino).
utente(666667777, joana, (21, 12, 1988), feminino).
utente(111222333, silvio, xpto1, masculino).
utente(555666777, gabriel, (17, 2, 2000), xpto3).
utente(456123, mariana, (18, 7, 2002), feminino).

true.
```

Figura 66 – Alteração do sexo do utente com *id* “456123”.

11.2 Predicado *alterar_nome_utente* de um utente

De modo a poder alterar o nome de um utente, foi necessária a extensão do predicado evidenciado na figura 67.

```
alterar_nome_utente(IdUt,Nome):- involucao(utente(IdUt,_, (Dia, Mes, Ano),Sexo)),
evolucao(utente(IdUt, Nome , (Dia, Mes, Ano), Sexo)).
```

Figura 67 – Extensão do predicado *alterar_nome_utente*.

Desta forma, o conhecimento pode evoluir, por exemplo, de imperfeito incerto para perfeito.

A título de exemplo, no registo do utente com *id* “1245”, nascido a 30-05-2000, do sexo *masculino* não foi possível conhecer o seu nome. Mais tarde, obteve-se a informação que este se chamava *Marcelo*, figura 68.

```
?- evolucao(utente(1245,xpto99,(30,5,2000),masculino))
true .

?- listing(utente).
:- dynamic utente/4.

utente(123456780, antonio, (30, 6, 1990), masculino).
utente(987654321, beatriz, (30, 7, 1985), feminino).
utente(135246978, carlos, (30, 8, 1987), masculino).
utente(1245, xpto99, (30, 5, 2000), masculino).

true.
```

Figura 68 – Evolução do utente com id “1245”.

Sendo necessária a alteração do registo, utilizou-se o *alterar_nome_utente*, inserindo como argumentos o id do utente e o nome para o qual se deseja alterar, figura 69.

Verificou-se que foi possível alterar um conhecimento do tipo incerto para um conhecimento perfeito positivo.

```
?- alterar_nome_utente(1245,marcelo).
true .

?- listing(utente).
:- dynamic utente/4.

utente(123456780, antonio, (30, 6, 1990), masculino).
utente(987654321, beatriz, (30, 7, 1985), feminino).
utente(135246978, carlos, (30, 8, 1987), masculino).
utente(1245, marcelo, (30, 5, 2000), masculino).

true.
```

Figura 69 – Alteração do nome do utente com id “1245”.

11.3 Predicado *alterar_data_ato*

Para se alterar a data do registo de um ato, foi necessária a extensão do predicado evidenciado na figura 70.

```
alterar_data_ato(IdAto, (Dia, Mes, Ano)):- involucao(ato(IdAto,_, IdUt,Idade, Colesterol, Pulsacao, PInf, PSup)),
evolucao(ato(IdAto,(Dia, Mes, Ano), IdUt,Idade, Colesterol, Pulsacao, PInf, PSup)).
```

*Figura 70 – Extensão do predicado *alterar_data_ato*.*

Por exemplo, pretende-se alterar a data em que o ato com *id* “*gmr2183*” foi criado. Para tal, utilizou-se o *alterar_data_ato*, inserindo como argumentos o id do ato e a data do ato para a qual se deseja alterar. Na figura 71 verifica-se o sucesso desta alteração de informação.


```
?- listing(ato).
:- dynamic ato/8.

ato(gmr02067, (30, 6, 2020), 123456780, 30, 140, 70, 60, 123).
ato(gmr2183, (30, 7, 2021), 987654321, 36, 190, 60, 70, 142).
ato(gmr2297, (30, 6, 2022), 123456780, 32, 230, 90, 65, 151).
ato(gmr1111, (12, 9, 2020), 111133333, 18, 260, 90, 50, 80).
ato(gmr22223, (1, 1, 2019), 222223333, 30, 160, 170, 200, 80).
ato(gmr22223, (2, 3, 2018), 333334444, 28, 140, 120, 80, 310).
ato(gmr02000, (18, 11, 2022), 4545456666, 30, xpto2, 70, 60, 123).

true.

?- alterar_data_ato(gmr2183, (01.01.1990)).
true.

?- listing(ato).
:- dynamic ato/8.

ato(gmr02067, (30, 6, 2020), 123456780, 30, 140, 70, 60, 123).
ato(gmr2297, (30, 6, 2022), 123456780, 32, 230, 90, 65, 151).
ato(gmr1111, (12, 9, 2020), 111133333, 18, 260, 90, 50, 80).
ato(gmr22223, (1, 1, 2019), 222223333, 30, 160, 170, 200, 80).
ato(gmr22223, (2, 3, 2018), 333334444, 28, 140, 120, 80, 310).
ato(gmr02000, (18, 11, 2022), 4545456666, 30, xpto2, 70, 60, 123).
ato(gmr2183, (1, 1, 1990), 987654321, 36, 190, 60, 70, 142).

true.
```

Figura 71 – Alteração da data de um ato com id “gmr2183”.

11.5 Predicado *alterarnaoutente*

Tendo em conta a evolução do conhecimento, nomeadamente a alteração de conhecimento negativo para positivo, foi desenvolvido um predicado que permitisse introduzir a informação negativa existente para um utente, e evoluísse a base de conhecimento com informação positiva do utente. Na figura 72, pode observar-se que inicialmente, o predicado faz a involução do conhecimento negativo introduzido e existente na base de conhecimento, e, posteriormente, evolui com o mesmo conhecimento, mas em termos positivos.

```
alterarnaoutente(-utente(IdUt, Nome, (Dia, Mes, Ano), Sexo)):- involucao(-utente(IdUt, Nome, (Dia, Mes, Ano), Sexo)),
evolucao(utente(IdUt, Nome, (Dia, Mes, Ano), Sexo)).
```

Figura 72 – Predicado *alterarnaoutente*.

Para exemplificar, mostra-se a seguir que após a utilização deste predicado, a utente *Carolina*, deixa de ter o seu conhecimento negativo, verificado pela *listingnaoutente* e passa a ter o seu conhecimento positivo, verificado pelo *listing(utente)*.

```
?- alterarnaoutente(-utente(9999888777, carolina, (12, 4, 2001), feminino)).
true.

?- listing(utente).
:- dynamic utente/4.

utente(123456780, antonio, (30, 6, 1990), masculino).
utente(987654321, beatriz, (30, 7, 1985), feminino).
utente(135246978, carlos, (30, 8, 1987), masculino).
utente(111133333, raquel, (14, 10, 2002), feminino).
utente(222223333, sofia, (3, 7, 1989), feminino).
utente(333334444, helena, (4, 2, 1990), feminino).
utente(444445555, marta, (15, 3, 1995), feminino).
utente(555556666, diana, (13, 4, 2000), feminino).
utente(666667777, joana, (21, 12, 1988), feminino).
utente(111222333, silvio, xpto1, masculino).
utente(555666777, gabriel, (17, 2, 2000), xpto3).
utente(9999888777, carolina, (12, 4, 2001), feminino).

true.
```

Figura 73 – Teste do predicado *alterarnaoutente*.

11.6 Predicado *alterarutente*

Da mesma forma, tendo em conta a evolução do conhecimento positivo para negativo, desenvolveu-se o predicado *alterarutente*, mostrado na figura 74.

```
alterarutente(utente(IdUt, Nome, (Dia, Mes, Ano), Sexo)):- involucao(utente(IdUt, Nome, (Dia, Mes, Ano), Sexo)),
evolucao(-utente(IdUt, Nome, (Dia, Mes, Ano), Sexo)).
```

Figura 74 – Predicado *alterarutente*.

O raciocínio utilizado é semelhante. Inicialmente, o predicado retira o conhecimento positivo, relativo ao utente introduzido, existente na base de conhecimento. Posteriormente, faz a evolução desse mesmo conhecimento, mas em termos negativos.

Na figura 75, mostra-se o teste deste predicado. Através *listing(utente)* pode-se verificar que a utente Joana já não tem o seu conhecimento positivo e pelo *listingnaoutente* verifica-se que a utente passou a constar no conhecimento negativo da base de conhecimento.

```
?- alterarutente(utente(666667777, joana, (21, 12, 1988), feminino)).
true.

?- listingnaoutente(L).
L = [utente(1928238723, julio, (22, 4, 1990), masculino), utente(9999888777, carolina, (12, 4, 2001), feminino), utente(666667777, joana, (21, 12, 1988), feminino)].

?- listing(utente).
:- dynamic utente/4.

utente(123456780, antonio, (30, 6, 1990), masculino).
utente(987654321, beatriz, (30, 7, 1985), feminino).
utente(135246978, carlos, (30, 8, 1987), masculino).
utente(111133333, raquel, (14, 10, 2002), feminino).
utente(222223333, sofia, (3, 7, 1989), feminino).
utente(333334444, helena, (4, 2, 1990), feminino).
utente(444445555, marta, (15, 3, 1995), feminino).
utente(555556666, diana, (13, 4, 2000), feminino).
utente(111222333, silvio, xptol, masculino).
utente(555666777, gabriel, (17, 2, 2000), xpto3).

true.
```

Figura 75 – Teste do predicado *alterarutente*.

12. Relacionar análises e atos com o utente

Foram desenvolvidos predicados que permitissem listar as informações pretendidas de um utente, como por exemplo, todos os atos existentes para esse utente na parte positiva da base de conhecimento ou ainda o historial de colesterol, pulsação e pressão arterial.

12.1 Predicado *relatar_atos_utente*

Este predicado, mostrado na figura 76, lista todos os atos de um utente quando introduzidas as suas informações. Assim, através do *findall*, devolve uma lista com todos os atos encontrados para esse utente.

```
relatar_atos_utente(utente(IdUt, Nome, Data, Sexo), L):- findall((ato(IdAto, DataAto, IdUt, Idade, Col, Pul, PInf, PSup))),
(ato(IdAto, DataAto, IdUt, Idade, Col, Pul, PInf, PSup)), L).
```

Figura 76 – Predicado *relatar_atos_utente*.

Na figura 77, pode observar-se a listagem dos atos relativos ao utente António de “id 123456780”.

```
?- relatar_atos_utente(utente(123456780, antonio, (30, 6, 1990), masculino), L).
L = [ato(gmr02067, (30, 6, 2020), 123456780, 30, 140, 70, 60, 123), ato(gmr2297, (30, 6, 2022), 123456780, 32, 230, 90, 65, 151)].
```

Figura 77 – Teste do predicado relatar_atos_utente.

12.2 Predicados listagemcolesterol, listagempulsacao e listagempressao

Com o mesmo raciocínio utilizado para o predicado *relatar_atos_utente*, foram desenvolvidos os predicados *listagemcolesterol*, *listagempulsacao* e *listagempressao* mostrados nas figuras 78, 79 e 80, respetivamente. Estes predicados mostram uma listagem do historial de cada parâmetro para o utente introduzido.

```
listagemcolesterol(utente(IdUt, Nome, Data, Sexo), L):-findall((DataAto, Col),
                                                             (ato(IdAto, DataAto, IdUt, Idade, Col, Pul, PInf, PSup)), L).
```

Figura 78 – Predicado listagemcolesterol.

```
listagempulsacao(utente(IdUt, Nome, Data, Sexo), L):-findall((DataAto, Pul),
                                                             (ato(IdAto, DataAto, IdUt, Idade, Col, Pul, PInf, PSup)), L).
```

Figura 79 – Predicado listagempulsacao.

```
listagempressao(utente(IdUt, Nome, Data, Sexo), L):-findall((DataAto, (PInf, PSup)),
                                                            (ato(IdAto, DataAto, IdUt, Idade, Col, Pul, PInf, PSup)), L).
```

Figura 80 – Predicado listagempressao.

Na figura 81, pode observar-se que quando testados os predicados mostrados acima no SWI, obtém-se, para cada um dos predicados uma lista com os valores obtidos do parâmetro pretendido bem como a data na qual o utente obteve esses valores. Por outras palavras, os predicados devolvem uma lista com as datas dos atos e o valor do parâmetro pretendido.

```
?- listagemcolesterol(utente(123456780, antonio, (30, 6, 1990), masculino), L).
L = [((30, 6, 2020), 140), ((30, 6, 2022), 230)].

?- listagempulsacao(utente(123456780, antonio, (30, 6, 1990), masculino), L).
L = [((30, 6, 2020), 70), ((30, 6, 2022), 90)].

?- listagempressao(utente(123456780, antonio, (30, 6, 1990), masculino), L).
L = [((30, 6, 2020), 60, 123), ((30, 6, 2022), 65, 151)].
```

Figura 81 – Testes dos predicados listagemcolesterol, listagempulsacao e listagempressao.

13. Marcadores

De forma a relacionar o valor obtido do parâmetro pulsação nos atos existentes na base de conhecimento e a informação do marcador que estabelece os valores normais desta análise, foi

desenvolvido um predicado, que lista as informações dos utentes que tem atos com o valor da análise pulsação dentro dos valores normais.

Este predicado da figura 82 verifica se a idade do utente na data do ato se encontra dentro das idades padrão e, por fim, se o valor, neste caso, pulsação, se encontra nos valores pretendidos.

```
dentropulsacao(L):- findall((IdUt, Nome, DataA, Pul),
    ((ato(_, DataA, IdUt, Idade, _, Pul, _, _)), (Pul>=60, Pul<=80, Idade=<45, Idade>=30,
    (utente(IdUt, Nome, Data, masculino)))),
    L).
```

Figura 82 – Predicado dentropulsacao.

De forma contrária, o predicado *forapulsacao*, na figura 83 lista os utentes que tem atos com valores de pulsação fora dos valores normais.

```
forapulsacao(L):- findall((IdUt, Nome, DataA, Pul),
    ((ato(_, DataA, IdUt, Idade, _, Pul, _, _)), (Pul<60; Pul>80), Idade=<45, Idade>=30,
    (utente(IdUt, Nome, Data, masculino)))),
    L).
```

Figura 83 – Predicado forapulsacao.

Como se pode observar na figura 84, os predicados listam os utentes que na data do ato tiveram a análise de pulsação dentro e fora, respetivamente, dos valores ditos normais.

```
?- dentropulsacao(L).
L = [(123456780, antonio, (30, 6, 2020), 70)].

?- forapulsacao(L).
L = [(123456780, antonio, (30, 6, 2022), 90)].
```

Figura 84 – Teste dos predicados dentropulsacao e forapulsacao.

O mesmo raciocínio poderia ter sido feito para os outros parâmetros existentes nos atos, como, por exemplo, o colesterol, pressão arterial inferior e superior. No entanto, é necessário ter em conta que as soluções inseridas na lista mostrada devem ser procuradas na parte positiva da base de conhecimento.

15. Conclusões

Neste trabalho, abordou-se a questão de representação de conhecimento positivo, negativo e desconhecido utilizando a linguagem em PROLOG. Para uma maior fiabilidade dos dados apresentados no mundo das análises clínicas, foram geradas invariantes de modo a promover algumas inconsistências na base de conhecimento.

Utilizando a linguagem PROLOG como ferramenta de representação e manipulação de dados, incluiu-se dois tipos de negação, a negação por falha (não) e a negação forte ($\neg$). Neste contexto, passou a ser possível distinguir situações falsas de situações para as quais não existe prova de que sejam verdadeiras ou falsas.

Introduzindo regras para a formalização do pressuposto do mundo fechado, conseguiu-se representar outro tipo de conhecimento, o conhecimento imperfeito desconhecido, caracterizados por 3 tipos de valores nulos: incerto, impreciso e interdito.

Foi possível interpretar o conhecimento imperfeito recorrendo ao uso do meta-predicado *si*, desenvolvido num programa em lógica, bem como os predicados *siL*, *siC*, *siD*.

Foi também gerado o predicado *alterar* para determinados parâmetros de utentes e atos, caso seja necessário alterar algum valor por algum motivo. Por outro lado, teve-se em conta a evolução do conhecimento, implementando-se predicados que permitissem alterar conhecimento positivo para negativo e vice-versa.

Finalmente, foi realizada a listagem de exames, como por exemplo, colesterol, pulsação por utente e por data, bem como a listagem de atos por utente e a sua relação com os marcadores existentes.

16. Referências Bibliográficas

[Analide, Neves] Analide, C., Neves, J.
“Representação de Informação Incompleta”,
Texto Pedagógico, Departamento de Informática, Universidade do Minho, Braga,
Portugal.